МПК-2025.01

G06N 3/06

G06N 3/08

B25J 9/16

G06T 7/20

G05B 13/00

Method for Adaptive Robot Learning Using Motion Capture Devices and Cloud-Based Neural Network Training

The present invention relates to the field of robotic systems, and more specifically to methods for robot learning based on human demonstrations, including motion capture and cloud-based data processing.

There are known systems in which a robot is programmed using motion capture suits and human demonstrations, followed by solving inverse kinematics problems. Other sources describe cloud architectures for robot training, employing distributed processing and real-time scalability. However, the described solutions are limited either by a fixed connection between the operator and the robot (e.g., low-level demonstration without dynamic learning), or they apply cloud computing to isolated tasks (such as SLAM – simultaneous localization and mapping), but not in the context of demonstration-based learning with automatic model updates for the robot.

The disadvantages of existing solutions include the lack of a unified architecture that integrates a motion capture suit, cloud-based imitation learning, and an automatic model update mechanism for the robot. There is no concrete solution for ensuring a continuous "data stream" encompassing capture, transmission, training, validation, and deployment. Furthermore, there is no specification of methods for secure data transmission, synchronous processing of demonstrations, and model quality verification prior to deployment.

The invention is known from patent CN 119188692A, titled "A Method for Training Humanoid Robot Movement Based on Limb Motion Recognition Using Wearable Devices." The essence of the method lies in training humanoid robot movements based on limb motion recognition, characterized by the use of a full-body dynamic motion capture device based on MEMS technology, and includes the following steps: S1. Motion training engineers wear full-body motion capture equipment based on MEMS technology. S2. A wireless connection is established between the full-body MEMS-based motion capture device and the humanoid robot. S3. The motion training engineer performs exercises, and the MEMS-based dynamic full-body motion capture device collects all movement data of the engineer. S4. The full-body dynamic motion capture device transmits all captured motion data in real time to the humanoid robot’s

low-level joint control system. S5. The lower-level computer controls each joint, making it move in accordance with the rotation data of the corresponding joint. S6. The joint control system stores the motion data in a motion database and uses a Large Language Model (LLM) to train on the motion data in order to create the humanoid robot's motor neural center system.

The drawbacks of the known method include:

- The use of the robot itself for data collection limits training to pre-defined exercises and scenarios only, restricting flexibility and scalability;
- There is no collection of tactile data, which is essential for performing more complex and adaptive tasks such as object manipulation or interaction with the environment;
- The absence of an automatic update mechanism for transferring newly acquired skills to the robot after neural network training limits the robot's ability to refine its actions and adapt to environmental changes or new tasks. This makes the robot learning process less dynamic and constrains its potential for autonomous improvement.

A known method is "Motion Capture: Teaching Humanoid Robots Human-Like Movement" developed by Movella [https://www.movella.com/resources/cases/humanoid-robots-learning-human-movement-using-xsens-motion-capture]. The essence of the method lies in the use of Xsens motion capture technology to teach humanoid robots human-like movements. Engineers wear Xsens motion capture suits, which capture all their movements in real time. These data are transmitted to a system that allows robots to reproduce the recorded motions, enabling them to "learn" human actions. The motion data are processed using artificial intelligence, helping robots adapt and refine their movements to perform tasks autonomously. The learning process involves repeated practice of movements and real-time corrections to improve precision. The Xsens technology ensures high-accuracy motion capture, and the system allows for the simultaneous training of multiple robots, accelerating the learning process. The combination of motion capture and AI enables the development of humanoid robots that can perform tasks accurately and efficiently across various industries, particularly in fields requiring process automation.

The drawbacks of the known method include:

- Lack of tactile data collection, which is essential for performing more complex and adaptive actions such as object manipulation or interaction with the environment;
- No automatic update of the robot with newly acquired skills after neural network training, which limits the robot's ability for self-improvement and adaptation to new tasks;

- Absence of structured data, which hinders effective neural network training, reducing its reliability and adaptability.

A known work is the article “A Cloud-based Robot System for Long-term Interaction: Principles, Implementation, Lessons Learned” [https://doi.org/10.1145/3481585]. The essence of the article is a comprehensive study of the PAL system (Personal Assistant for a Healthy Lifestyle), designed for long-term adaptive interaction between humans and robots. The system uses a cloud-based architecture, enabling scalability, remote data processing, and continuous learning, which ensures its ability to evolve and meet user needs over time. The system’s modular design supports customization, allowing it to be adapted for various use cases such as healthcare and well-being, where ongoing interaction is essential. The integration of hybrid AI technologies—including machine learning, natural language processing, and computer vision—enables the robot to deliver context-aware responses, learn from interactions, and adapt to user preferences and environments. The system also uses a standardized interaction model, simplifying communication and making it intuitive regardless of the user's technical background. During the implementation of PAL, the article highlights key lessons, such as: the importance of designing for long-term interaction, the value of modularity for customization and updates, and the need for continuous adaptation based on user data. These principles not only enhance the user experience but also ensure that the robot remains relevant and effective in supporting the user, especially in applications related to health and well-being.

The drawbacks of the known technical solution include:

- Lack of tactile data collection, which is essential for performing more complex and adaptive tasks such as object manipulation or interaction with the environment.
- Limited training capability, restricted to pre-defined exercises and scenarios, since data collection is performed using the robot itself.
- Absence of structured data, which hinders the effective training of neural networks, thereby reducing their reliability and adaptability.

The identified analogues coincide with the claimed technical solution only in certain individual features. Therefore, a prototype has not been identified, and the claims of the invention are formulated without a limiting (restrictive) part.

The technical problem addressed by the present invention, and its technical result, is the development of a Method for Adaptive Learning of a Robot Using Motion Capture Devices and Cloud-Based Neural Network Training, which, in comparison with existing analogues, provides the following advantages:

- Execution of more complex and adaptive actions, such as object manipulation or interaction with the environment, enabled by the collection of tactile data;
- Expanded training capabilities beyond predefined exercises and scenarios, made possible by data collection without relying on the robot itself;
- High quality of structured data, ensuring more stable and adaptive neural network training, achieved through time synchronization of all data sources.

The essence of the claimed technical solution is a Method for Adaptive Learning of a Robot Using Motion Capture Devices and Cloud-based Neural Network Training, **which comprises:** which consists in that the operator wears a real-time human motion capture suit equipped with motion capture sensors, as well as gloves with tactile sensors on the fingers; in addition to the suit, the operator sets up cameras for recording objects; all measured parameters are synchronized by timestamps; the recording device on the suit stores data packets containing: a unique sensor identifier defining its location on the suit, a timestamp, kinematic parameters, pressure readings applied to the fingertips, and video frames; upon completion of the data collection session, all accumulated kinematic, tactile, and video data are transmitted to a remote server via a secure communication channel; the received data are processed on the server using neural network models, and an updated version of the model is generated based on the demonstration movements; after meeting predefined quality criteria, the trained model is packaged into a versioned artifact; then, the software deployment system, triggered upon the appearance of a new model version, automatically publishes the updated version of the neural network model to a secure repository storage; after successful publication, the system sends notifications to all registered robots, informing them about the updated model version and providing metadata, including the new model version and the download address; a robot, functioning autonomously and connected to the internet, periodically sends a request to check for model updates; if an update is available, the robot downloads the model via a secure protocol; the model is then launched in an isolated environment, where an automatic set of tests is executed – including synthetic movements, trajectory compliance checks, and tactile condition verification; if the testing is successful, the model is activated: it is launched, connected to the robot controllers, and control is handed over to the neural network; the robot then performs operations corresponding to the processed demonstration movements of the operator.

The following is a description of the claimed technical solution provided by the applicant.

The claimed Method for Adaptive Learning of a Robot Using Motion Capture Devices and Cloud-based Neural Network Training consists of the following sequence of actions.

1. The operator wears a real-time human motion capture suit equipped with motion capture sensors (such as inertial IMU sensors, optical markers, mechanical encoders, etc.), as well as gloves with tactile sensors on the fingers (such as strain gauges, piezoresistive elements, etc.) capable of measuring the pressure applied to the fingertips. The suit and gloves provide the capture of kinematic data of the human body parts and tactile readings (pressure applied to the fingers), synchronizing the data using timestamps. In addition to the suit, the operator mounts cameras on the body in the area of the head and hands to record the objects with which the operator interacts, and the video frames are synchronized with the sensor module data using identical timestamps.

2. The suit includes a built-in recording device that stores data packets containing: a unique sensor identifier specifying its location on the suit, a timestamp, kinematic parameters (position, orientation, angular velocity, acceleration), pressure readings applied to the fingers collected via tactile sensors, and video frames recorded from the cameras.

3. Upon completion of the data collection session, all accumulated kinematic, tactile, and video data are transmitted to a remote server via a secure communication channel (for example, using the TLS — Transport Layer Security — protocol), ensuring the confidentiality and integrity of the transmitted information.

4. The received data are processed on the server using neural network models to generate an updated version of the model based on the demonstration movements. For this purpose, a cloud platform is deployed on the server, implementing an end-to-end ML pipeline for training a Visual–Tactile–Navigation–Language–Action model.

For example, the process may include the following stages:

– Reception of raw data – incoming data include timestamps, sensor identifiers, kinematic parameters (e.g., position, orientation, angular velocity, and acceleration), tactile readings from the fingers, and video frames from the cameras;

– Preprocessing and filtering – recursive filters (e.g., Kalman filter) and low-pass filters are applied to remove noise and smooth signals. Normalization and alignment – data are scaled to a unified range, and signals from different sensors are synchronized using timestamps, forming uniform time series;

– Batch preparation – batch aggregation is performed, creating training batches that contain ordered multi-modal input data (visual tensors, tactile signals, navigation information, language labels) for input into the model;

– Neural network training – the data are processed through a multimodal architecture designed to train a policy capable of handling visual–tactile streams, understanding language commands, and generating navigation and motor actions;

– Model generation – after achieving the specified quality metrics, the model is formatted and prepared for deployment into the robotic system.

After reaching predefined quality criteria (e.g., accuracy and robustness metrics), the trained model is packaged into a versioned artifact (for example, a Docker image). At the packaging stage, the artifact is assigned a digital signature (e.g., using PKI certificates, encoded with the developer's private key and a SHA-256 checksum), which ensures integrity and authenticity guarantees.

5. Next, the software deployment system (e.g., Jenkins, GitLab CI, or similar), triggered upon the appearance of a new model version, automatically publishes the updated version of the neural network model to a secure repository storage (such as a container registry or file storage with HTTPS access and authentication via OAuth or PKI certificates). After successful publication, the system sends notifications (e.g., webhooks or MQTT) to all registered robots, informing them about the updated model version and providing metadata, including the new model version (e.g., SHA-256 hash) and the download address.

6. The robot, operating autonomously and connected to the internet, periodically sends a request to check for model updates. If an update is available, the robot downloads the model (e.g., via HTTPS) using a secure protocol—receiving an accompanying hash and PKI signature—and then performs integrity and authenticity verification (validating the signature using the publisher’s PKI certificates). The model is then launched in an isolated environment (e.g., a sandbox or container), where an automated test suite is executed—this includes synthetic motion sequences, trajectory compliance checks, and tactile condition validation.

7. If the testing is successful, the model is activated: it is launched (directly participating in the generation of control commands), connected to the robot's controllers, and control is handed over to the neural network.

The robot then proceeds to perform operations corresponding to the processed demonstration movements of the operator.

The following is an example of the implementation of the claimed technical solution.

1. The operator wore an Xsens (Movella) motion capture suit for real-time human motion tracking, equipped with inertial IMU sensors, as well as ergoGLOVE gloves with tactile sensors on the fingers, capable of measuring the pressure applied to the fingertips. The suit and gloves provided the capture of kinematic data from the human body parts and tactile readings from the fingers, with all data synchronized via timestamps. In addition to the suit, the operator mounted Intel RealSense D435 cameras on the body near the head and wrists to record the objects the operator interacted with. The video frames recorded by the cameras were synchronized with the sensor module data using identical timestamps.

2. The suit included an external HDD as a recording device, which stored data packets containing: a unique sensor identifier specifying its location on the suit, a timestamp, kinematic parameters (position, orientation, angular velocity, acceleration), pressure readings applied to the fingertips collected via tactile sensors, and video frames recorded by the cameras.

3. Upon completion of the data collection session, all accumulated kinematic and tactile data packets were transmitted to a remote server using the TLS (Transport Layer Security) protocol, ensuring the confidentiality and integrity of the transmitted information.

4. The received data were processed on the server using neural network models to generate an updated version of the model based on the demonstration movements. For this purpose, a cloud platform was deployed on the server, implementing an end-to-end ML pipeline for training a Visual–Tactile–Navigation–Language–Action model.

The process included the following stages:

– Timestamps, sensor identifiers, kinematic parameters (position, orientation, angular velocity, and acceleration), tactile readings from the fingers, as well as video frames from the cameras were received;

– A Kalman filter and a low-pass filter were applied to remove noise and smooth the signals. The data were normalized to a unified scale, and signals from different sensors were synchronized using timestamps, forming uniform time series;

– Training batches were formed, containing ordered multi-modal input data: visual tensors, tactile signals, navigation information, and language labels, for feeding into the model;

– The data were processed through a multimodal architecture designed to train a policy capable of handling visual–tactile streams, understanding language commands, and generating navigation and motor actions;

– After achieving the specified quality metrics, the model was formatted and prepared for deployment into the robotic system.

After achieving the specified accuracy and robustness metrics, the trained model was packaged into a Docker image. At the packaging stage, the artifact was assigned a digital signature using PKI certificates, encoded with the developer's private key, and included a SHA-256 checksum, ensuring guarantees of integrity and authenticity.

5. Next, the Jenkins software deployment system, triggered by the appearance of a new model version, automatically published the updated version of the neural network model to a container registry. After successful publication, the system sent MQTT push notifications to all registered robots, informing them about the updated model version and providing metadata, including the SHA-256 hash version and the download URL.

6. The robot, operating autonomously and connected to the internet, periodically sent a request to check for model updates. If an update was available, the robot downloaded the model via a secure protocol, received the accompanying hash and PKI signature, and then performed integrity and authenticity verification. The model was then launched in a container, where an automated test suite was executed—this included synthetic movements, trajectory compliance checks, and tactile condition validation.

7. If the testing was successful, the model was activated: it was launched (directly participating in the generation of control commands), connected to the robot's controllers, and control was handed over to the neural network.

Thus, from the above description, **it can be concluded that the applicant has successfully addressed the identified technical problem and achieved the claimed technical result**, namely the development of a Method for Adaptive Learning of a Robot Using Motion Capture Devices and Cloud-Based Neural Network Training, which, in comparison with existing analogues, provides the following advantages:

– Execution of more complex and adaptive actions, such as object manipulation and interaction with the environment, made possible by the collection of tactile data;

– Extended training capabilities beyond predefined exercises and scenarios, enabled by data collection independent of the robot itself;

– High-quality structured data, ensuring stable and adaptive neural network training, achieved through synchronization of all sensor and video data via timestamps.

Claim of the Invention

A method for adaptive learning of a robot using motion capture devices and cloud-based neural network training, characterized by the following sequence of actions:

The operator wears a real-time human motion capture suit equipped with motion capture sensors, as well as gloves with tactile sensors on the fingers. In addition to the suit, the operator mounts cameras to record the objects being interacted with. All measured parameters are synchronized using timestamps.

The recording device on the suit stores data packets containing: a unique sensor identifier specifying its location on the suit, a timestamp, kinematic parameters, pressure readings applied to the fingertips, and video frames.

Upon completion of the data collection session, all accumulated kinematic, tactile, and video data are transmitted to a remote server via a secure communication channel.

The received data are processed on the server using neural network models, resulting in the generation of an updated version of the model based on the demonstration movements. After meeting predefined quality criteria, the trained model is packaged into a versioned artifact.

Next, the software deployment system, triggered by the appearance of a new model version, automatically publishes the updated version of the neural network model to a secure repository. After successful publication, the system sends notifications to all registered robots, informing them about the updated model version and providing metadata, including the new model version and the download address.

The robot, operating autonomously and connected to the internet, periodically sends a request to check for model updates. If an update is available, the robot downloads the model via a secure protocol, then launches the model in an isolated environment, where an automated test suite is executed — including synthetic movements, trajectory compliance checks, and tactile condition validation.

If the testing is successful, the model is activated: it is launched, connected to the robot's controllers, and control is handed over to the neural network. The robot then proceeds to perform operations corresponding to the processed demonstration movements of the operator.

ABSTRACT

Method for Adaptive Learning of a Robot Using Motion Capture Devices and Cloud-Based Neural Network Training

The present invention relates to the field of robotic systems, and more specifically to methods for robot learning based on human demonstrations, including motion capture and cloud-based data processing.

The essence of the invention is a method for adaptive learning of a robot using motion capture devices and cloud-based neural network training, **which comprises the following:** The operator wears a real-time human motion capture suit equipped with motion capture sensors, as well as gloves with tactile sensors on the fingers. In addition to the suit, the operator mounts cameras to record the objects being interacted with. All measured parameters are synchronized using timestamps. The recording device integrated into the suit stores data packets containing: a unique sensor identifier specifying its location on the suit, a timestamp, kinematic parameters, pressure readings applied to the fingertips, and video frames. Upon completion of the data collection session, all accumulated kinematic, tactile, and video data are transmitted to a remote server via a secure communication channel. The received data are processed on the server using neural network models to generate an updated version of the model based on the demonstration movements. After meeting predefined quality criteria, the trained model is packaged into a versioned artifact. Next, a software deployment system, triggered by the appearance of a new model version, automatically publishes the updated version of the neural network model to a secure repository. After successful publication, the system sends notifications to all registered robots, informing them about the updated model and providing metadata, including the new model version and the download address. A robot, functioning autonomously and connected to the internet, periodically checks for model updates. If an update is available, the robot downloads the model via a secure protocol, then launches the model in an isolated environment, where it executes an automated test suite, including synthetic movements, trajectory compliance checks, and tactile condition verification. If the testing is successful, the model is activated: it is launched, connected to the robot's controllers, and control is handed over to the neural network. The robot then performs operations corresponding to the processed demonstration movements of the operator.

Examiner: F.R. Khabibullin